

上节回顾

- 分页管理：非连续分配，离散分布在内存不同位置
- 页面与物理块
 - 页是程序逻辑空间上的划分
 - 块是内存物理地址上的划分
 - 两者大小相等，通过页表一一对应
- 分页的地址结构：页号+页内地址
- 页表：记录每个页对应的物理块
- 分页地址变换过程
 - CPU给出逻辑地址
 - OS分割为页号和段号
 - OS查页表从页号得到块号
 - 页内地址等于块内地址，两者合并得到物理地址
- 逻辑地址转换为物理地址的计算方法
- 带快表的分页：地址变换过程

基本分页存储管理方式

■ 单级页表的缺陷：

- ❑ 页表一般是数组的形式，空间分配属于连续分配
- ❑ 页表大小不能超过单个物理块大小
- ❑ 当逻辑地址空间很大时，导致页表占用内存空间很大
- ❑ 例如地址长度**32**位，页表大小**4KB**，可以得到最大空间为**4GB**，最多包含 **$4G/4K=1M$** 个页

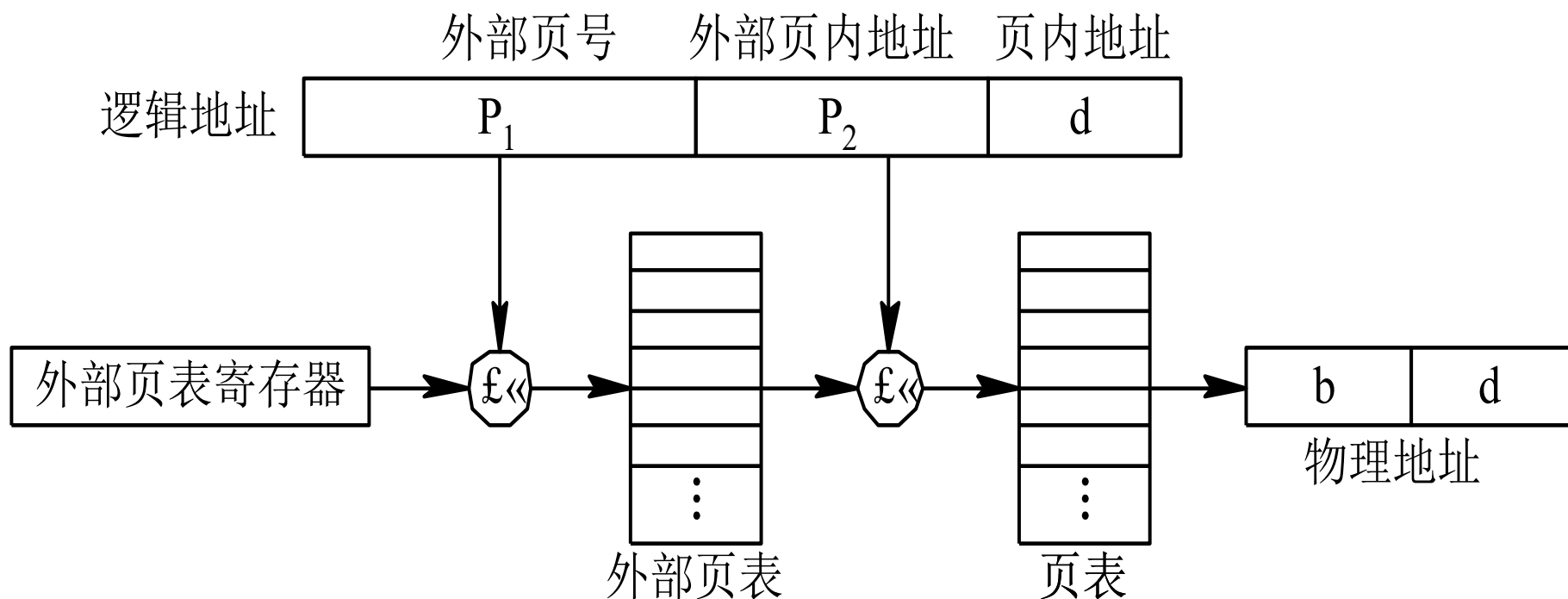
若每个页表项占用**4**个字节，则页表最大长度为**4MB**，即要求内存划分出连续**4MB**的空间来装载页表，而且页表采用的是连续分配，不是分页分配

■ 解决方法

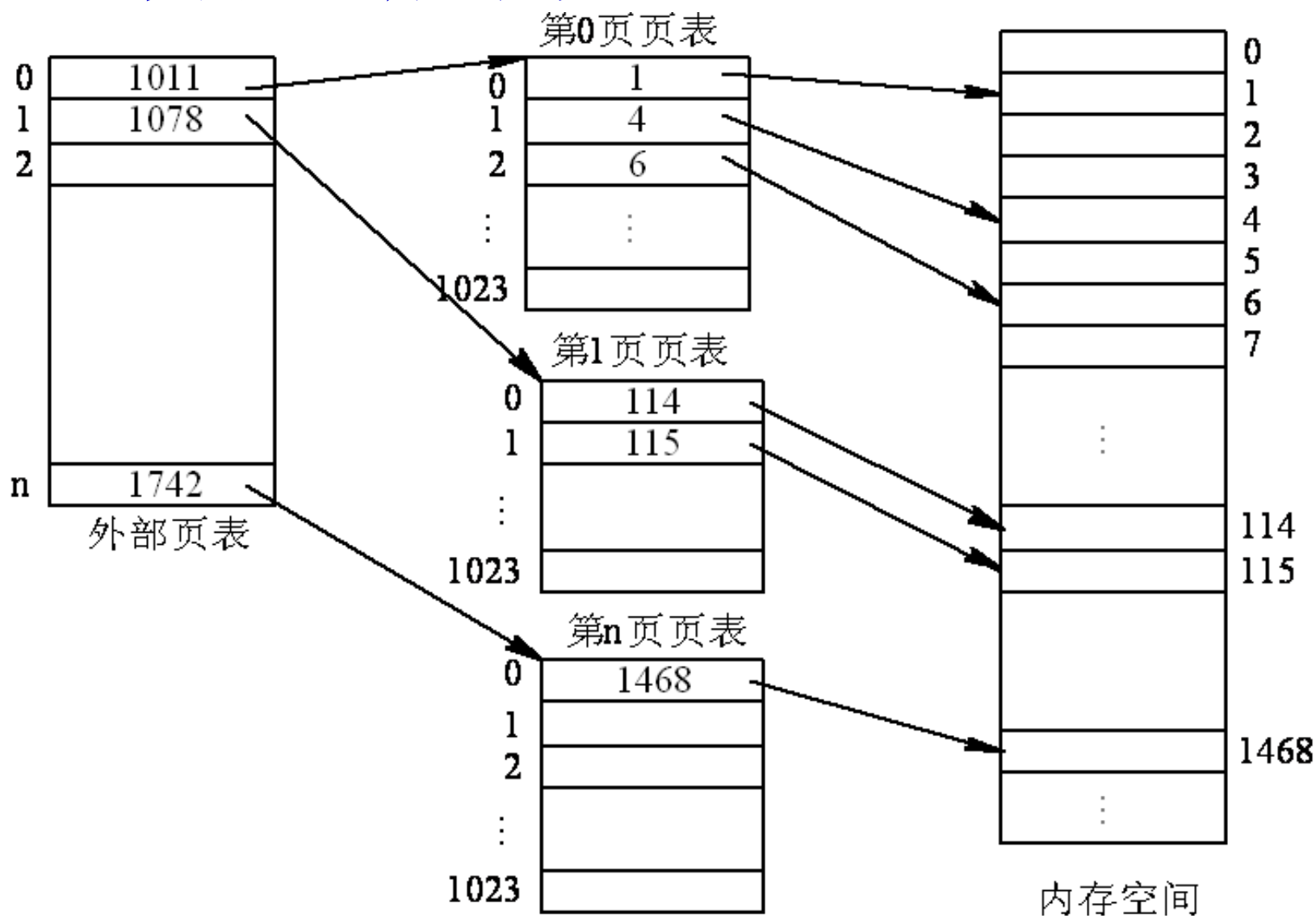
- ❑ 采用离散分配方式的管理页表
- ❑ 将当前需要的部分页表项调入内存，其余的页表项仍驻留在磁盘上，需要时再调入

基本分页存储管理方式

- 两级页表：对页表本身采用分页式管理，对页表本身增加了一层页表管理
- 两级页表的逻辑地址结构与地址变换



基本分页存储管理方式



基本分页存储管理方式

■ 多级页表

□ 32位系统采用两级页表结构是合适的

□ 对于64位系统，页表大小4KB

则页内地址为12位，外部页内地址也为12位，外部页号则有42位，即最多可以有4096G个页表项，实际并不需要这么多

□ 64位系统最大支持空间 $2^{64} = 1844744 \text{ TB}$ ，目前实际只使用48位，采用三级页表就可以了

练习

- 有一基本分页存储管理系统，假定页面大小为**1024**字节，页表采用离散方式放在内存中。每个页表项占用**4**个字节。在忽略页表及系统区等占用空间的条件下：
 - 若采用一级页表，系统能支持的逻辑空间最多为多少？
 - 若采用二级页表，系统能支持的逻辑空间最多为多少？
 - 在二级页表中，现有逻辑地址为**542783**，求该地址对应的外层页号，外层页内地址和页内地址？
 - 注意题目中所有数据都是十进制

基本分段存储管理方式

■ 基本概念

- 分段是指将一个进程划分为多个具有逻辑意义而且相对独立的部分
- 例如主程序段、数据段、堆栈段、子程序段等
- **X86常见的段基址：CS、SS、DS、ES**
- 每个段都是从**0**开始的独立逻辑地址空间

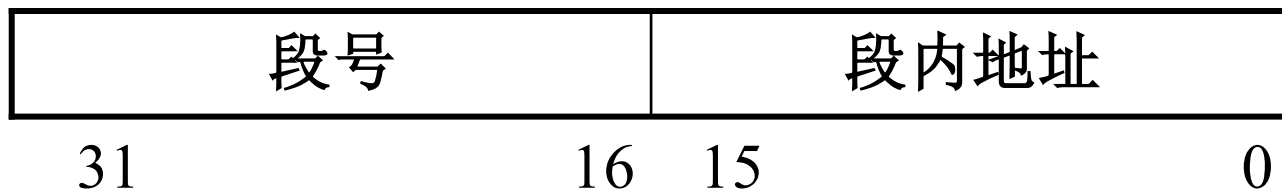
■ 引入分段的好处

- 方便编程 ·
- 信息共享
- 信息保护
- 动态增长
- 动态链接

基本分段存储管理方式

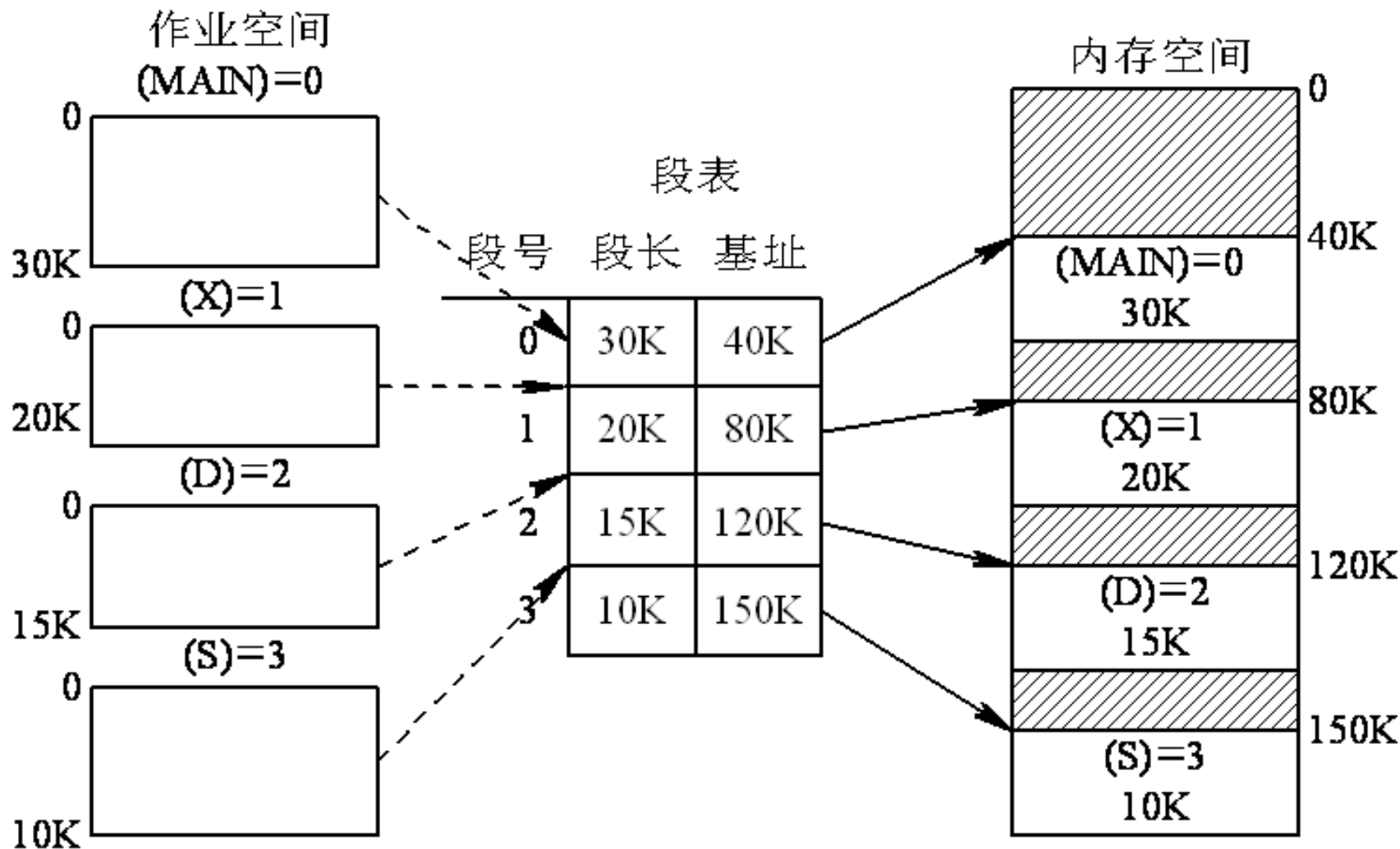
■ 分段的地址结构

- 每个段都有独立的名称，为了方便，用段号表示
- 在逻辑地址意义上，每个段地址包含两部分
- 所以段地址是二维结构



基本分段存储管理方式

段表

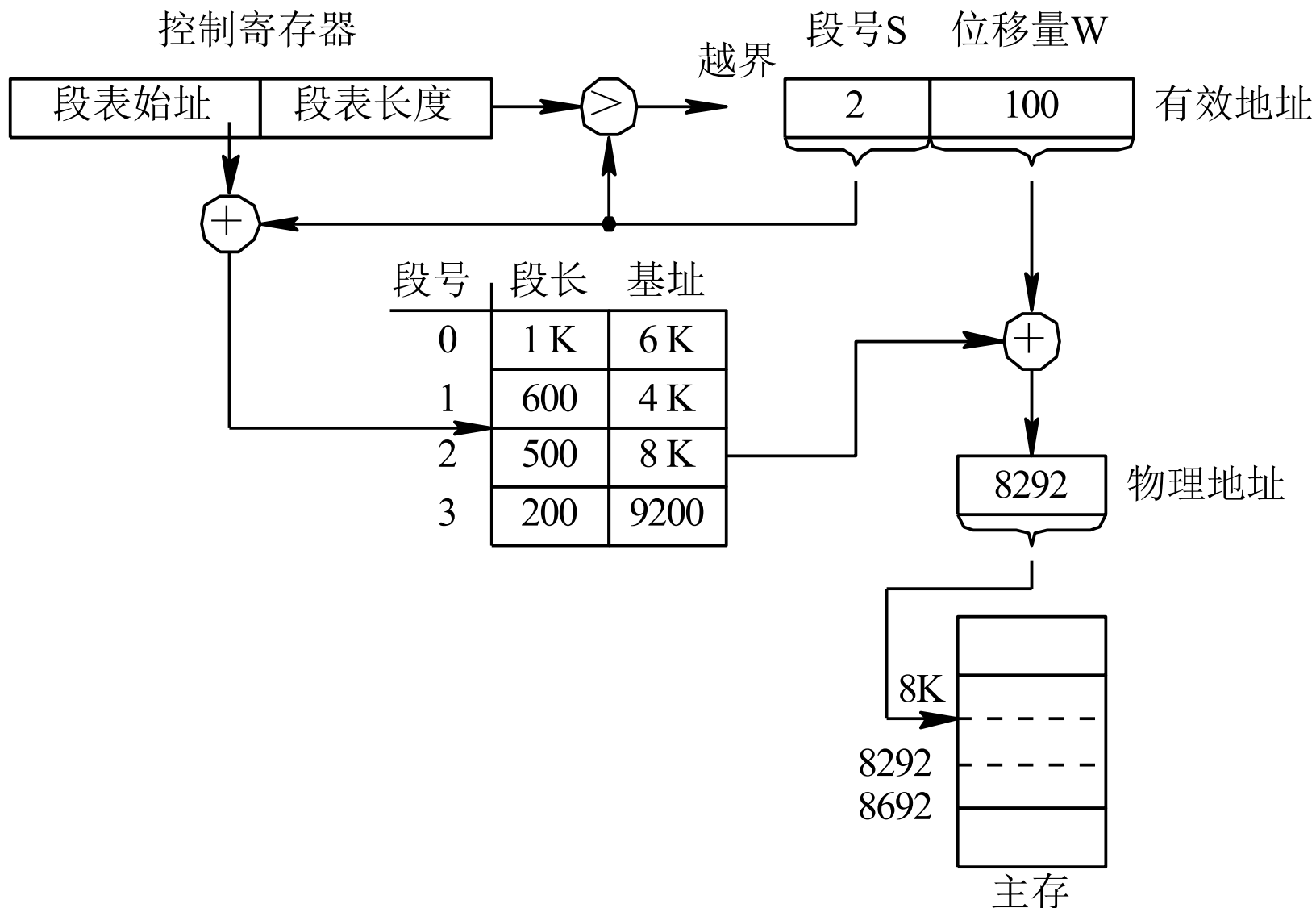


基本分段存储管理方式

■ 段地址变换过程

- ① 进程访问某个逻辑地址时，先取得段号
- ② 若段号大于段表长度，越界错误
- ③ 查段表得到段开始地址和段长度
- ④ 若段内地址大于段长度，越界错误
- ⑤ 通过段开始地址和段内地址得到物理地址
- ⑥ 根据物理地址读取数据

基本分段存储管理方式



基本分段存储管理方式

- 具有快表的分段地址变换
 - 将段表放入快表中
 - 由于段的数量远少于页的数量，段表装入快表的百分比要比页表大，所以分段的快表命中率也高，一般比常规内存访问慢**10%~15%**

基本分段存储管理方式

■ 分页与分段的区别

□ 页是信息的物理单位，满足系统空间管理需要。分页实现离散分配方式，以消减内存的外零头，提高内存的利用率。

段是信息的逻辑单位，满足用户需要

□ 页的大小固定，由系统决定

段的长度不固定，由用户决定

□ 分页的作业地址空间是一维的

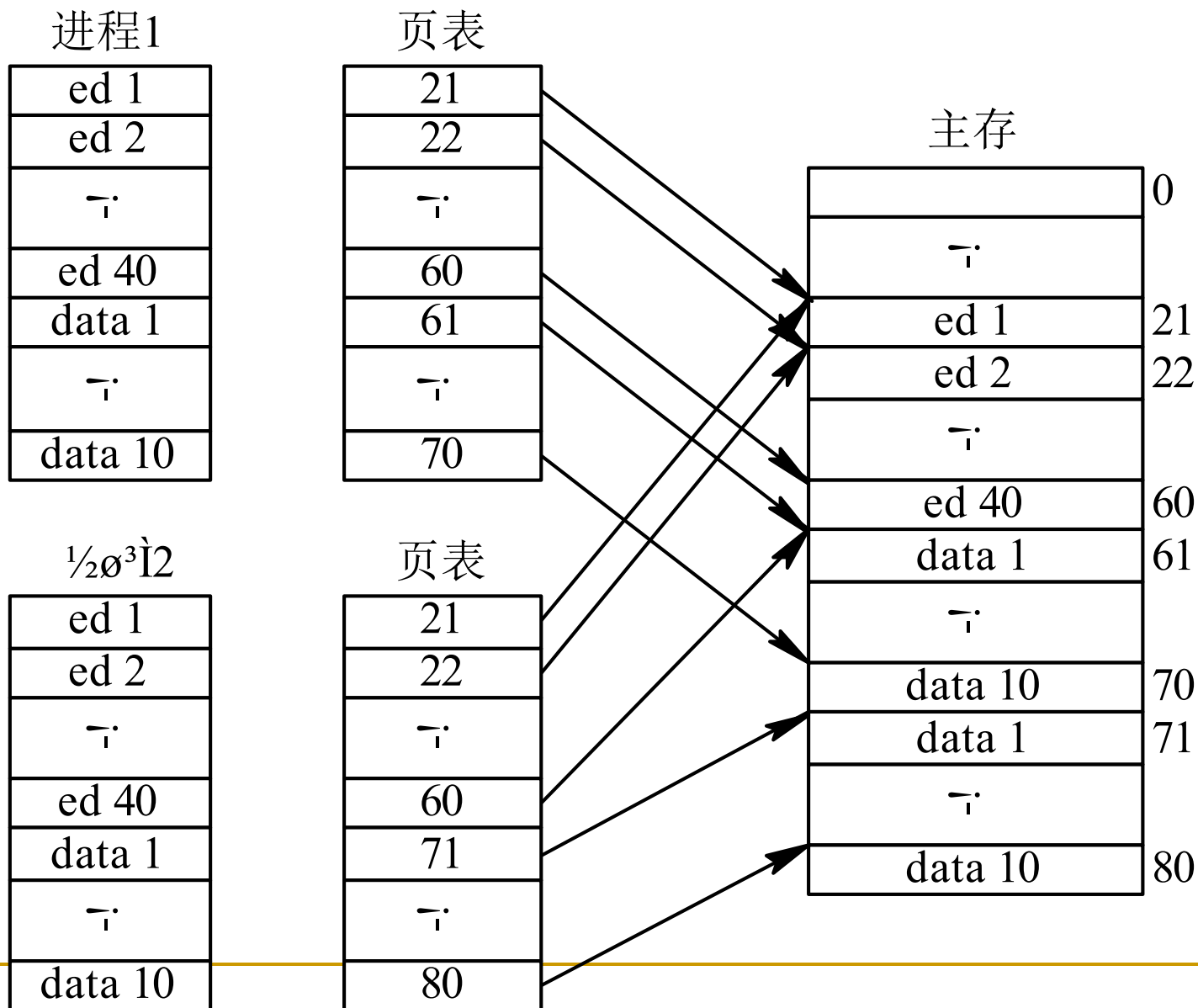
分段的作业地址空间则是二维的

基本分段存储管理方式

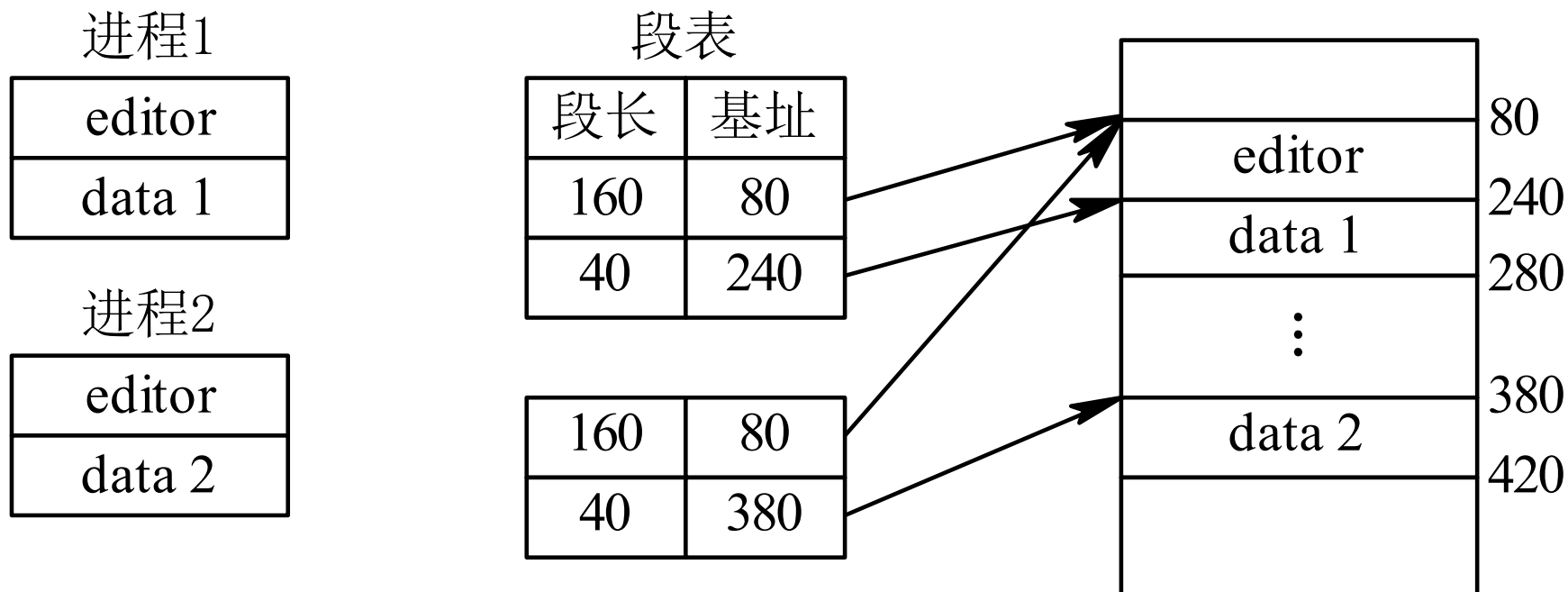
■ 信息共享

- 段是有意义的信息逻辑单位，含有一组相对完成独立的信息
- 多个进程共享一个具有独立意义的段，实现很简单，只要在各自段表加入相同段基址和段长度就可以了
- 例如**Windows**动态链接库中的纯代码段，可以被多个进程共享

基本分段存储管理方式



基本分段存储管理方式



练习

- 某系统采用分段式存储管理，在一个进程中，它的段表如下所示，求逻辑地址为（**2**，**895**）对应的物理地址（所有数据都是十进制）

段号	开始地址
0	80K
1	12K
2	25K
3	72K

基本分段存储管理方式

■ 段页式管理的引入

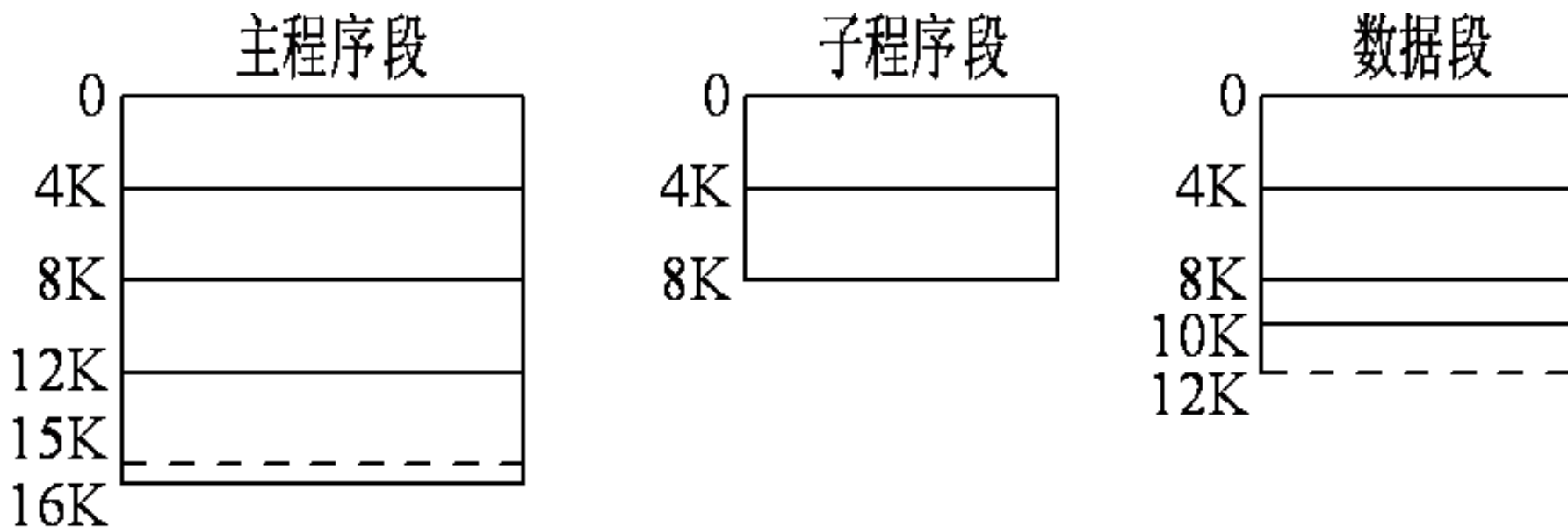
- ❑ 段面向用户程序需要，段长度不固定
- ❑ 段需要连续分配空间，存在连续分配的缺点，例如易产生碎片
- ❑ 结合段式和页式两者管理优点，既能节省内存空间，提高内存分配效率；又能兼顾用户程序需要

■ 段页式管理的机制

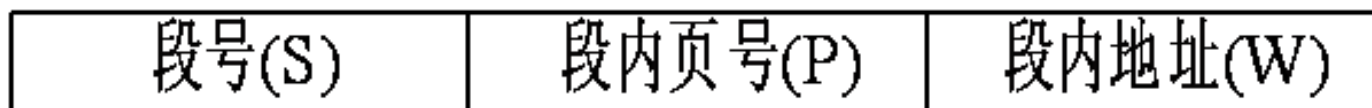
- ❑ 分段与分页机制的结合
- ❑ 先将用户程序划分为多个有逻辑意义的段，再将段划分为多个页
- ❑ 段页式管理需要设置段表和页表
- ❑ 每个段都对应一张页表，因此段表存放了每张页表的开始地址和页表长度

基本分段存储管理方式

■ 段页式管理的地址结构



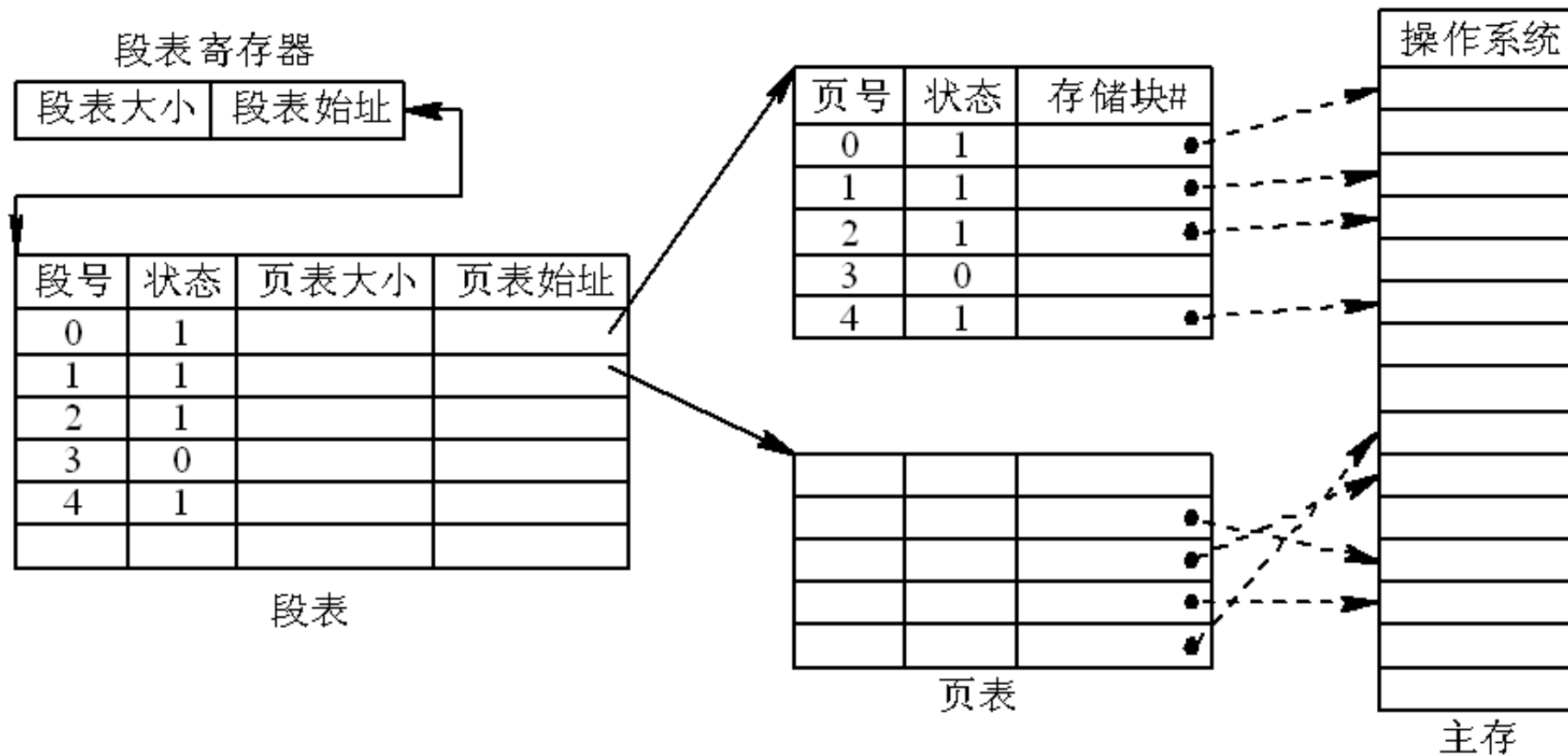
(a)



(b)

基本分段存储管理方式

■ 段页式的地址映射



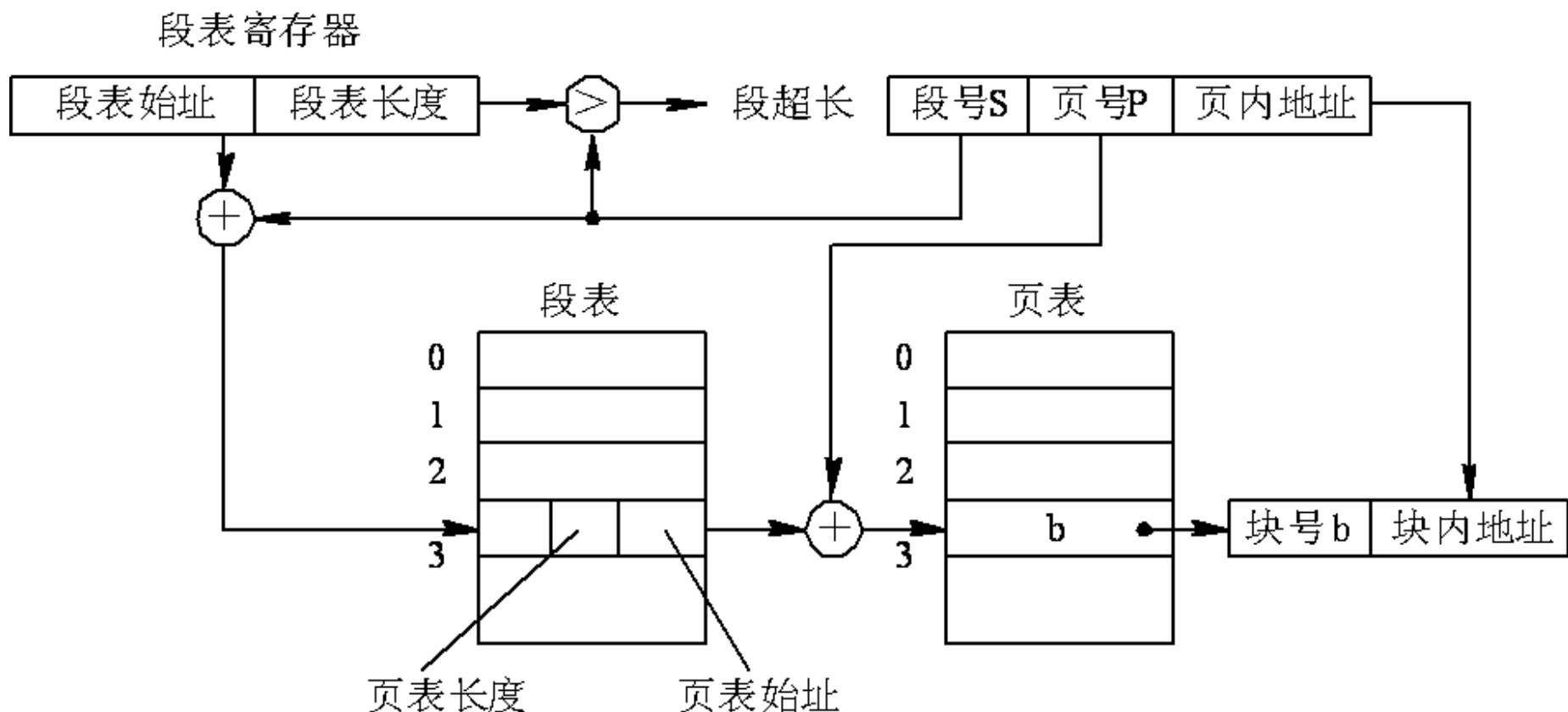
基本分页存储管理方式

■ 段页式的地址变换过程

- ① 进程访问某个逻辑地址时，先取得段号
- ② 段号大于段表长度，越界错误
- ③ 查段表，得到页表开始地址
- ④ 在页表内，根据段内页号查物理块号
- ⑤ 页内地址直接对应块内地址
- ⑥ 通过物理块号和块内地址得到物理地址
- ⑦ 根据物理地址读取数据

基本分段存储管理方式

■ 段页式的地址变换过程



基本分页存储管理方式

■ 段页式管理中的快表

- 原有的地址变换需要访问三次内存：段表+页表+数据
- 增加一个高速缓冲寄存器，每次访问以段号和页号为索引去检索高速缓存，从中找到相应的物理块号。
- 若找不到，再执行三次访问。

练习

- 在一个段页式存储管理系统，页面大小为**1024**字节。在一个进程中，它的段表、页表如下所示，求逻辑地址为（**1**，**2099**）对应的物理地址。

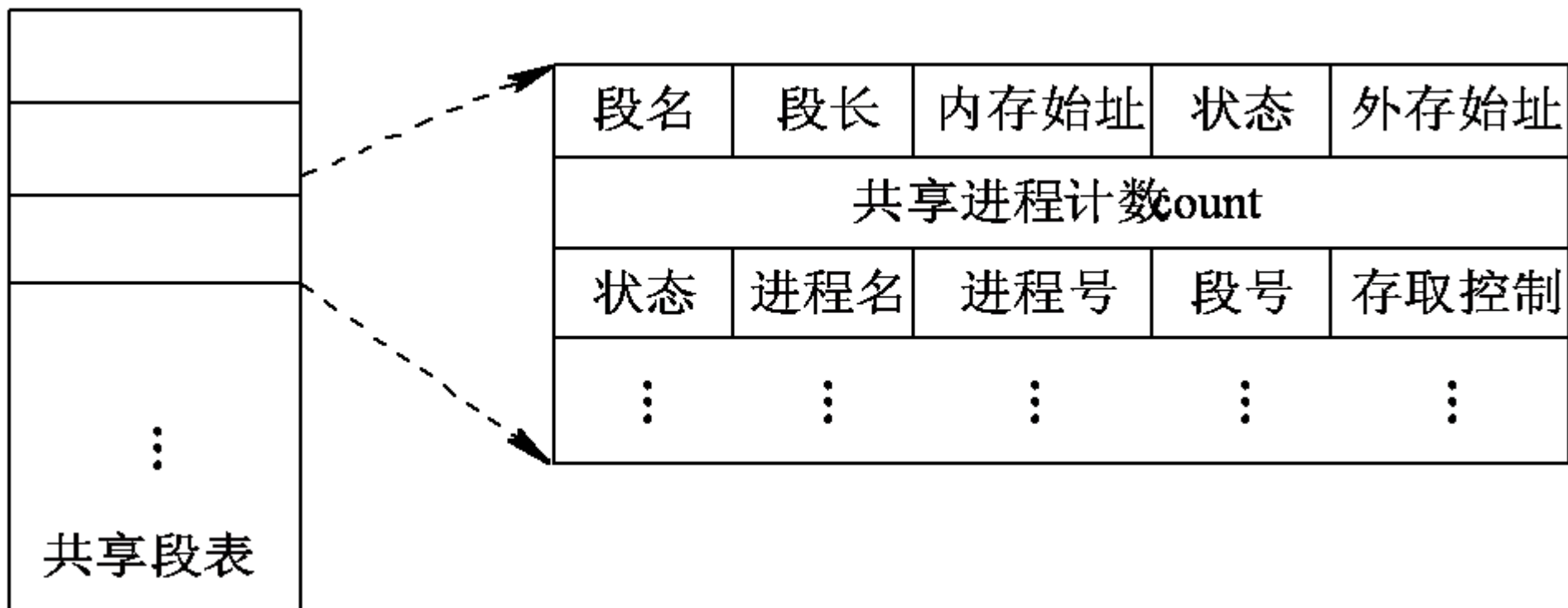
段表 (第2块)			段内页表 (第76块)			段内页表 (第33块)			段内页表 (第52块)	
段号	页表首址		页号	内存块号		页号	内存块号		页号	内存块号
0	76		0	18		0	65		0	6
1	33		1	22		1	13		1	98
2	52		2	36		2	40		2	82

请求分段存储管理方式

分段的共享与保护

■ 共享段表

- 共享进程计数**count**: 记录共享进程数, 影响段回收操作
- 存取控制字段: 记录访问权限, 文件主、其他进程
- 段号: 不同进程可以使用不同段号共享同一段



请求分段存储管理方式

■ 共享段的分配

- ❑ 为共享段分配空间，调入共享段
- ❑ 首个调用共享段的进程段表中填写记录
- ❑ 在共享段表中增加一项，填写信息，
- ❑ 设**count**为1
- ❑ 若有其他进程调用共享段，无需再分配空间
- ❑ 在调用进程段表中增加表项，填写共享段的物理地址
- ❑ 在共享段表中填上调用进程的进程名、存取控制等，
- ❑ **count+1**

请求分段存储管理方式

■ 共享段的回收

- 当某进程不需要共享段
- 删除该进程段表中对应共享的表项
- 删除该进程在共享段表中的有关记录
- **count-1**
- 如果**count**为0，系统回收该共享段的物理内存，取消共享段表，此时已没有进程使用该段

■ 共享段的特点

- 由第一个调用共享段的进程请求系统分配空间
- 由最后一个调用共享段的进程请求系统回收空间

请求分段存储管理方式

■ 分段保护

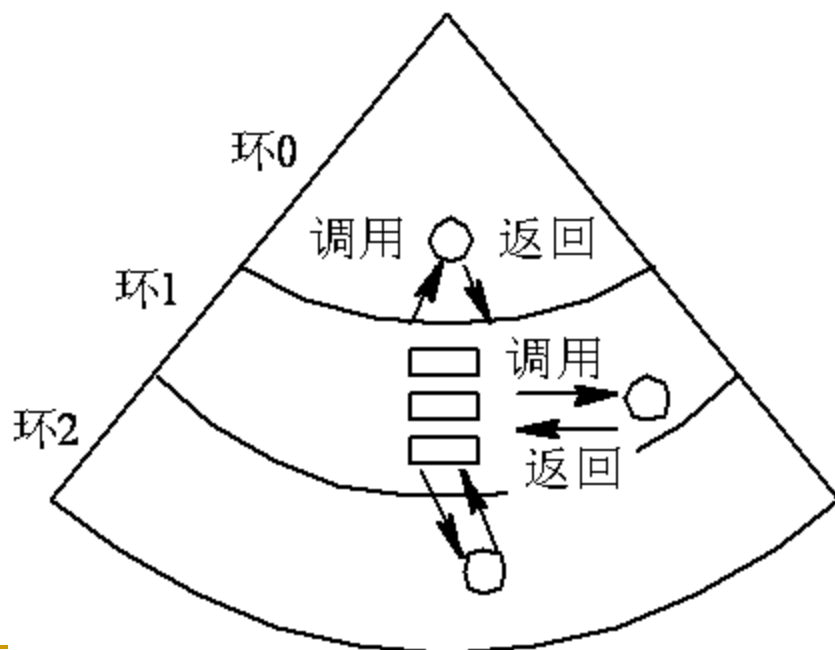
□ 越界检查

□ 存取控制检查：只读、只执行、读/写

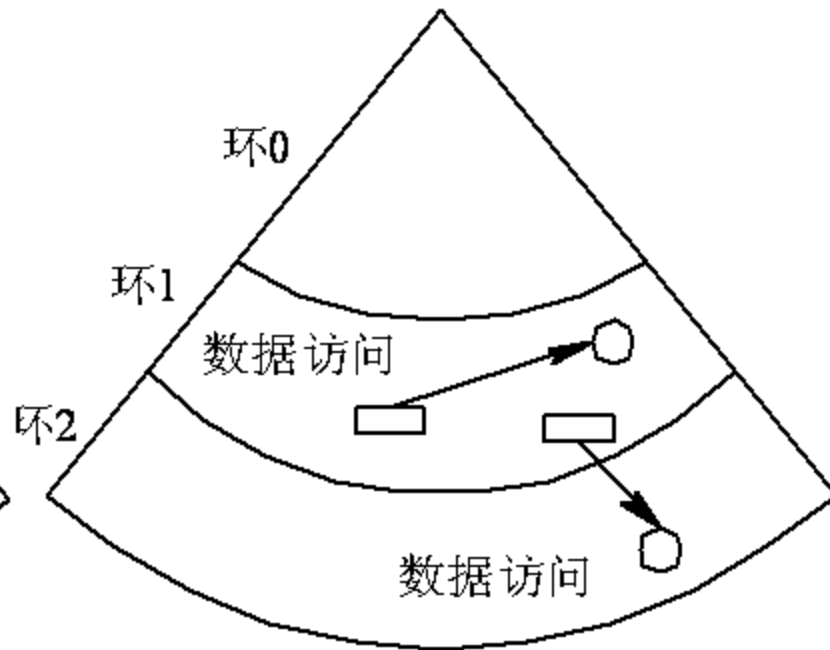
□ 环保护机构

■ 程序可以访问驻留在相同环或较低特权环中的数据

■ 程序可以调用驻留在相同环或较高特权环中的服务



(a) 程序间的控制传输



(b) 数据访问